

Documento:Actividad final del Módulo III

Tarea: Base de Datos sobre clientes

Capturas De Pantalla: De Comandos Ejecutados

Nombre De Autores

Dina Arely Tino De Martínez

Marlon Gerardo Flores Portillo

Universidad Doctor Andres Bello Reguinal Sonsonate.

Nombre: Del Módulo.

Módulo III : Bases De Datos No Relacionales

Catedrático

Ingeniero:Jose Rosalio Serrano

Entrega,Sábado,30 /05/2026

1)Desarrollo de Actividad final del Módulo III

Indicaciones

- Tener múltiples direcciones.
- Realizar múltiples compras.
- Guardar productos favoritos.
- Registrar múltiples métodos de pago.
- Generar historial de navegación.

Captura de pantalla sobre Base de Datos Sobre clientes

Análisis :¿Qué hace?

1. Crea un bucle que inserta 5000 documentos uno por uno
2. Cada cliente tiene arrays para direcciones, compras, métodos de pago.

MONGOSH

```
use clientesDB
```

```
switched to db clientesDB
```

```
db["clientes"].find().for(let i=0; i<5000; i++){
```

```
  db.clientes.insertOne({
```

```
    nombre: "Cliente " + i,
```

```
    email: "cliente" + i + "@mail.com",
```

```
    telefono: "7000-" + Math.floor(Math.random()*10000),
```

```
    direcciones: [
```

```
      {tipo: "casa", calle: "Calle " + i, ciudad: "San Salvador", pais: "SV"},
```

```
      {tipo: "trabajo", calle: "Blvd " + i, ciudad: "San Salvador", pais: "SV"}]
```

```
  ],
```

```
  compras: [
```

```
    {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
```

```

    productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]

  ],
  compras: [
    {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
      productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
  ],
  productosFavoritos: ["P" + i, "P" + (i+1)],
  metodosPago: [
    {tipo: "tarjeta", ultimos4: "1234"},
    {tipo: "paypal", email: "cliente" + i + "@mail.com"}
  ],
  historialNavegacion: [
    {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
  ]
})
}

```

```
> mongosh: Clientes  Clientes  clientes  > mongosh: Clientes  +

>_MONGOSH

> for(let i=0; i<5000; i++){
  db.clientes.insertOne({
    nombre: "Cliente " + i,
    email: "cliente" + i + "@mail.com",
    telefono: "7000-" + Math.floor(Math.random()*10000),
    direcciones: [
      {tipo: "casa", calle: "Calle " + i, ciudad: "San Salvador", pais: "SV"},
      {tipo: "trabajo", calle: "Blvd " + i, ciudad: "San Salvador", pais: "SV"}
    ],
    compras: [
      {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
        productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
    ],
    productosFavoritos: ["P" + i, "P" + (i+1)],
    metodosPago: [
      {tipo: "tarjeta", ultimos4: "1234"},
      {tipo: "paypal", email: "cliente" + i + "@mail.com"}
    ],
    historialNavegacion: [
      {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
    ]
  })
}
```

```
> mongosh: Clientes  Clientes  clientes  > mongosh: Clientes  +

>_MONGOSH

< {
  acknowledged: true,
  insertedId: ObjectId('6a151a544cf5f304ad4e2999')
}
> for(let i=0; i<5000; i++){
  db.clientes.insertOne({
    nombre: "Cliente " + i,
    email: "cliente" + i + "@mail.com",
    telefono: "7000-" + Math.floor(Math.random()*10000),
    direcciones: [
      {tipo: "casa", calle: "Calle " + i, ciudad: "San Salvador", pais: "SV"},
      {tipo: "trabajo", calle: "Blvd " + i, ciudad: "San Salvador", pais: "SV"}
    ],
    compras: [
      {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
        productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
    ],
    productosFavoritos: ["P" + i, "P" + (i+1)],
    metodosPago: [
      {tipo: "tarjeta", ultimos4: "1234"},
      {tipo: "paypal", email: "cliente" + i + "@mail.com"}
    ],
    historialNavegacion: [
      {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
    ]
  })
}
```

```
>_MONGOSH
compras: [
  {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
    productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
],
productosFavoritos: ["P" + i, "P" + (i+1)],
metodosPago: [
  {tipo: "tarjeta", ultimos4: "1234"},
  {tipo: "paypal", email: "cliente" + i + "@mail.com"}
],
historialNavegacion: [
  {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
]
})
}

< {
  acknowledged: true,
  insertedId: ObjectId('6a151abf4cf5f304ad4e3d21')
}
>
> db.clientes.countDocuments()
< 10000
clientesDB>
```

comprobando códigos ejecutados: para verificar:

```
db.clientes.countDocuments()db.clientes.find().limit(3).pretty()
```

```
>_MONGOSH
}
>
> db.clientes.countDocuments()
< 10000
> db.clientes.find().limit(3).pretty()
< {
  _id: ObjectId('6a151a484cf5f304ad4e1612'),
  nombre: 'Cliente 0',
  email: 'cliente0@mail.com',
  telefono: '7000-6062',
  direcciones: [
    {
      tipo: 'casa',
      calle: 'Calle 0',
      ciudad: 'San Salvador',
      pais: 'SV'
    },
    {
      tipo: 'trabajo',
      calle: 'Blvd 0',
      ciudad: 'San Salvador',
      pais: 'SV'
    }
  ],
  compras: [
```

```
>_MONGOSH
{
  fecha: 2026-05-26T03:58:00.065Z,
  total: 122.75,
  productos: [
    {
      id: 'P0',
      nombre: 'Prod0',
      precio: 50,
      cantidad: 1
    }
  ]
},
productosFavoritos: [
  'P0',
  'P1'
],
metodosPago: [
  {
    tipo: 'tarjeta',
    ultimos4: '1234'
  },
  {
    tipo: 'paypal',
    email: 'cliente0@mail.com'
  }
]
```

```
>_MONGOSH
metodosPago: [
  {
    tipo: 'tarjeta',
    ultimos4: '1234'
  },
  {
    tipo: 'paypal',
    email: 'cliente1@mail.com'
  }
],
historialNavegacion: [
  {
    producto: 'P1',
    fecha: 2026-05-26T03:58:00.093Z,
    tiempoSeg: 118
  }
]
}
{
  _id: ObjectId('6a151a484cf5f304ad4e1614'),
  nombre: 'Cliente 2',
  email: 'cliente2@mail.com',
  telefono: '7000-3265',
  direcciones: [
    {
```

```
>_ mongosh: C... x  Clientes  clientes  >_ mongosh: Clientes  +
>_MONGOSH
}
]
}
],
productosFavoritos: [
  'P2',
  'P3'
],
metodosPago: [
  {
    tipo: 'tarjeta',
    ultimos4: '1234'
  },
  {
    tipo: 'paypal',
    email: 'cliente2@mail.com'
  }
],
historialNavegacion: [
  {
    producto: 'P2',
    fecha: 2026-05-26T03:58:00.111Z,
    tiempoSeg: 225
  }
]
}
```

```
>_MONGOSH
}
],
productosFavoritos: [
  'P2',
  'P3'
],
metodosPago: [
  {
    tipo: 'tarjeta',
    ultimos4: '1234'
  },
  {
    tipo: 'paypal',
    email: 'cliente2@mail.com'
  }
],
historialNavegacion: [
  {
    producto: 'P2',
    fecha: 2026-05-26T03:58:00.111Z,
    tiempoSeg: 225
  }
]
}
clientesDB>
```

2)Consideraciones para la inserción de los datos :

- 5,000 registros mínimos.
- Cada cliente debe contener:
 - mínimo 3 direcciones,
 - 10 productos favoritos,
 - 5 métodos de pago,
 - 20 compras,
 - 30 registros de navegación.

Captura de pantalla Análisis

Qué hace:

1. Crea un bucle que inserta 5000 documentos uno por uno
2. Cada cliente tiene arrays para direcciones, compras, métodos de pago


```

for(let i=0; i<5000; i++){
  db.clientes.insertOne({
    nombre: "Cliente " + i,
    email: "cliente" + i + "@mail.com",
    telefono: "7000-" + Math.floor(Math.random()*10000),
    direcciones: [
      {tipo: "casa", calle: "Calle " + i, ciudad: "San Salvador", pais: "SV"},
      {tipo: "trabajo", calle: "Blvd " + i, ciudad: "San Salvador", pais: "SV"}
    ],
    compras: [
      {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
        productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
    ],
    productosFavoritos: ["P" + i, "P" + (i+1)],
    metodosPago: [
      {tipo: "tarjeta", ultimos4: "1234"},
      {tipo: "paypal", email: "cliente" + i + "@mail.com"}
    ],
    historialNavegacion: [
      {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
    ]
  })
}

```

```
mongosh: Clientes Clientes clientes mongosh: Clientes +
_MONGOSH
]
}
clientesDB> for(let i=0; i<5000; i++){
  db.clientes.insertOne({
    nombre: "Cliente " + i,
    email: "cliente" + i + "@mail.com",
    telefono: "7000-" + Math.floor(Math.random()*10000),
    direcciones: [
      {tipo: "casa", calle: "Calle " + i, ciudad: "San Salvador", pais: "SV"},
      {tipo: "trabajo", calle: "Blvd " + i, ciudad: "San Salvador", pais: "SV"}
    ],
    compras: [
      {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
        productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
    ],
    productosFavoritos: ["P" + i, "P" + (i+1)],
    metodosPago: [
      {tipo: "tarjeta", ultimos4: "1234"},
      {tipo: "paypal", email: "cliente" + i + "@mail.com"}
    ],
    historialNavegacion: [
      {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
    ]
  })
}
```

```
_MONGOSH
< {
  acknowledged: true,
  insertedId: ObjectId('6a151abf4cf5f304ad4e3d21')
}
>
> db.clientes.countDocuments()
< 10000
> db.clientes.find().limit(3).pretty()
< {
  _id: ObjectId('6a151a484cf5f304ad4e1612'),
  nombre: 'Cliente 0',
  email: 'cliente0@mail.com',
  telefono: '7000-6062',
  direcciones: [
    {
      tipo: 'casa',
      calle: 'Calle 0',
      ciudad: 'San Salvador',
      pais: 'SV'
    },
    {
      tipo: 'trabajo',
      calle: 'Blvd 0',
      ciudad: 'San Salvador',
```

```
>_MONGOSH

    pais: 'SV'
  }
],
compras: [
  {
    fecha: 2026-05-26T03:58:00.065Z,
    total: 122.75,
    productos: [
      {
        id: 'P0',
        nombre: 'Prod0',
        precio: 50,
        cantidad: 1
      }
    ]
  }
],
productosFavoritos: [
  'P0',
  'P1'
],
metodosPago: [
  {
    tipo: 'tarjeta',
    ultimos4: '1234'
  }
]
```

```
>_MONGOSH

    },
    {
      tipo: 'paypal',
      email: 'cliente0@mail.com'
    }
  ],
historialNavegacion: [
  {
    producto: 'P0',
    fecha: 2026-05-26T03:58:00.065Z,
    tiempoSeg: 202
  }
]
}
{
  _id: ObjectId('6a151a484cf5f304ad4e1613'),
  nombre: 'Cliente 1',
  email: 'cliente1@mail.com',
  telefono: '7000-4317',
  direcciones: [
    {
      tipo: 'casa',
      calle: 'Calle 1',
      ciudad: 'San Salvador',
      pais: 'SV'
    }
  ]
}
```

```

>_MONGOSH
},
{
  tipo: 'trabajo',
  calle: 'Blvd 1',
  ciudad: 'San Salvador',
  pais: 'SV'
}
],
compras: [
  {
    fecha: 2026-05-26T03:58:00.093Z,
    total: 19.92,
    productos: [
      {
        id: 'P1',
        nombre: 'Prod1',
        precio: 50,
        cantidad: 1
      }
    ]
  }
],
productosFavoritos: [
  'P1',
  'P2'
]

```

```

>_MONGOSH
    nombre: 'Prod1',
    precio: 50,
    cantidad: 1
  }
]
},
productosFavoritos: [
  'P2',
  'P3'
],
metodosPago: [
  {
    tipo: 'tarjeta',
    ultimos4: '1234'
  },
  {
    tipo: 'paypal',
    email: 'cliente2@mail.com'
  }
],
historialNavegacion: [
  {
    producto: 'P2',
    fecha: 2026-05-26T03:58:00.111Z,
    tiempoSeg: 225
  }
]

```

```

> MONGODB
historialNavegacion: [
  {
    producto: 'P2',
    fecha: 2026-05-26T03:58:00.111Z,
    tiempoSeg: 225
  }
]
}
}
> for(let i=0; i<5000; i++){
  db.clientes.insertOne({
    nombre: "Cliente " + i,
    email: "cliente" + i + "@mail.com",
    telefono: "7000-" + Math.floor(Math.random()*10000),
    direcciones: [
      {tipo: "casa", calle: "Calle " + i, ciudad: "San Salvador", pais: "SV"},
      {tipo: "trabajo", calle: "Blvd " + i, ciudad: "San Salvador", pais: "SV"}
    ],
    compras: [
      {fecha: new Date(), total: Math.round(Math.random()*200*100)/100,
        productos: [{id: "P" + i, nombre: "Prod" + i, precio: 50, cantidad: 1}]}
    ],
    productosFavoritos: ["P" + i, "P" + (i+1)],
    metodosPago: [
      {tipo: "tarjeta", ultimos4: "1234"},
      {tipo: "paypal", email: "cliente" + i + "@mail.com"}
    ]
  })
}
}
< {
  acknowledged: true,
  insertedId: ObjectId('6a152eda4cf5f304ad4e50a9')
}
clientesDB>

```

```

    ],
    historialNavegacion: [
      {producto: "P" + i, fecha: new Date(), tiempoSeg: Math.floor(Math.random()*300)}
    ]
  })
}
}
< {
  acknowledged: true,
  insertedId: ObjectId('6a152eda4cf5f304ad4e50a9')
}
clientesDB>

```

Verificado que funcionó:

db.clientes.count Documents() // Debe salir 5000

db.clientes.find().limit(1).pretty() // Revisa un documento con todos los arrays.

2)Consideraciones para la inserción de los datos :

Cada grupo deberá insertar:

- 5,000 registros mínimos.
- Cada cliente debe contener:

- mínimo 3 direcciones,
- 10 productos favoritos,
- 5 métodos de pago,
- 20 compras,
- 30 registros de navegación.

2) Solución escritura de códigos

```
db.Clientes.insertOne({
  nombre: "Cliente 1",
  email: "cliente1@mail.com",
  direcciones: [
    {tipo: "casa", calle: "Calle 1", ciudad: "San Salvador"},
    {tipo: "trabajo", calle: "Blvd 1", ciudad: "Soyapango"},
    {tipo: "envío", calle: "Col 1", ciudad: "Santa Tecla"}
  ],
  productosFavoritos: ["P1001", "P1002", "P1003", "P1004", "P1005",
"P1006", "P1007", "P1008", "P1009", "P1010"],
  metodosPago: [
    {tipo: "tarjeta", banco: "BAC"},
    {tipo: "paypal"},
    {tipo: "efectivo"},
    {tipo: "transferencia"},
    {tipo: "bitcoin"}
  ],
  compras: (function(){
```

```

    let arr = [];

    for(let j = 0; j < 20; j++){

        arr.push({idCompra: "C1" + j, total: 50 + j*15});

    }

    return arr;

})();

historialNavegacion: (function(){

    let arr = [];

    for(let j = 0; j < 30; j++){

        arr.push({producto: "P" + j, tiempoSeg: 60 + j*5});

    }

    return arr;

})();

})

db.Clientes.insertOne({

    nombre: "Cliente 1",

    email: "cliente1@mail.com",

    direcciones: [

        {tipo: "casa", calle: "Calle 1", ciudad: "San Salvador"},

        {tipo: "trabajo", calle: "Blvd 1", ciudad: "Soyapango"},

        {tipo: "envío", calle: "Col 1", ciudad: "Santa Tecla"}

    ],

    productosFavoritos: ["P1001", "P1002", "P1003", "P1004", "P1005",

"P1006", "P1007", "P1008", "P1009", "P1010"],

```

```

metodosPago: [
    {tipo: "tarjeta", banco: "BAC"},
    {tipo: "paypal"},
    {tipo: "efectivo"},
    {tipo: "transferencia"},
    {tipo: "bitcoin"}
],
compras: (function(){
    let arr = [];
    for(let j = 0; j < 20; j++){
        arr.push({idCompra: "C1" + j, total: 50 + j*15});
    }
    return arr;
})();
historialNavegacion: (function(){
    let arr = [];
    for(let j = 0; j < 30; j++){
        arr.push({producto: "P" + j, tiempoSeg: 60 + j*5});
    }
    return arr;
})();
})

```


TypeError: Cannot read properties of null (reading 'pretty')

```
db.Clientes.insertOne({
  nombre: "Cliente 1",
  email: "cliente1@mail.com",
  direcciones: [
    {tipo: "casa", calle: "Calle 1", ciudad: "San Salvador"},
    {tipo: "trabajo", calle: "Blvd 1", ciudad: "Soyapango"},
    {tipo: "envío", calle: "Col 1", ciudad: "Santa Tecla"}
  ],
  productosFavoritos: ["P1001", "P1002", "P1003", "P1004", "P1005",
"P1006", "P1007", "P1008", "P1009", "P1010"],
  metodosPago: [
    {tipo: "tarjeta", banco: "BAC"},
    {tipo: "paypal"},
    {tipo: "efectivo"},
    {tipo: "transferencia"},
    {tipo: "bitcoin"}
  ],
  compras: (function(){
    let arr = [];
    for(let j = 0; j < 20; j++){
      arr.push({idCompra: "C1" + j, total: 50 + j*15});
    }
    return arr;
  })
})
```

```

    })(),

    historialNavegacion: (function(){

        let arr = [];

        for(let j = 0; j < 30; j++){

            arr.push({producto: "P" + j, tiempoSeg: 60 + j*5});

        }

        return arr;

    })()

    })

    {

        acknowledged: true,

        insertedId: ObjectId('6a1a67d77dfa6ae657b75404')
    }

```

Captura de pantalla en mongoDB.

```

db.Clientes.insertOne({
  nombre: "Cliente 1",
  email: "cliente1@mail.com",
  direcciones: [
    {tipo: "casa", calle: "Calle 1", ciudad: "San Salvador"},
    {tipo: "trabajo", calle: "Blvd 1", ciudad: "Soyapango"},
    {tipo: "envío", calle: "Col 1", ciudad: "Santa Tecla"}
  ],
  productosFavoritos: ["P1001", "P1002", "P1003", "P1004", "P1005", "P1006", "P1007", "P1008", "P1009", "P1010"],
  metodosPago: [
    {tipo: "tarjeta", banco: "BAC"},
    {tipo: "paypal"},
    {tipo: "efectivo"},
    {tipo: "transferencia"},
    {tipo: "bitcoin"}
  ],
  compras: (function(){
    let arr = [];
    for(let j = 0; j < 20; j++){
      arr.push({idCompra: "C1" + j, total: 50 + j*15});
    }
  })()
})

```

```
for(let j = 0; j < 10; j++){
  arr.push({idCompra: "C1" + j, total: 50 + j*15});
}
return arr;
})();
historialNavegacion: (function(){
  let arr = [];
  for(let j = 0; j < 30; j++){
    arr.push({producto: "P" + j, tiempoSeg: 60 + j*5});
  }
  return arr;
})();
})()
{
  acknowledged: true,
  insertedId: ObjectId('6a1a67d77dfa6ae657b75404')
}
ClientesDB>
```

3) Consultas a realizar:

- Buscar clientes que tengan un producto favorito específico
- Buscar clientes que tengan compras mayores a \$300
- Contar clientes con más de 10 compras
- Agregar un nuevo producto favorito
- Eliminar un producto favorito
- Evitar elementos duplicados

Escritura de códigos

1. Buscar clientes que tengan un producto favorito específico:

```
db.Clientes.find({productosFavoritos: "P1003"}).pretty()
```

2. Buscar clientes que tengan compras mayores a \$300:

```
db.Clientes.find({"compras.total": {$gt: 300}}).pretty()
```

3. Contar clientes con más de 10 compras:

```
db.Clientes.find({$expr: {$gt: [{$size: "$compras"}, 10]}}).count()
```

4. Agregar un nuevo producto favorito - \$push:

```
db.Clientes.updateOne({email: "cliente1@mail.com"}, {$push:  
{productosFavoritos: "P9999"}})
```

```
db.Clientes.findOne({email: "cliente1@mail.com"}, {productosFavoritos: 1, _id:  
0})
```

5. Eliminar un producto favorito - \$pull:

```
db.Clientes.updateOne({email: "cliente1@mail.com"}, {$pull:  
{productosFavoritos: "P1005"}})
```

```
db.Clientes.findOne({email: "cliente1@mail.com"}, {productosFavoritos: 1, _id:  
0})
```

6. Evitar elementos duplicados - \$addToSet:

```
db.Clientes.updateOne({email: "cliente1@mail.com"}, {$addToSet:  
{productosFavoritos: "P8888"}})
```

```
db.Clientes.updateOne({email: "cliente1@mail.com"}, {$addToSet:  
{productosFavoritos: "P8888"}})
```

```
db.Clientes.findOne({email: "cliente1@mail.com"}, {productosFavoritos: 1, _id:  
0})
```

El segundo \$addToSet te debe dar modifiedCount: 0 - screenshot de eso prueba que no se duplicó.

Solución escritura de comandos ejecutados

```
{  
  
  {  
  
    producto: 'P14',  
  
    tiempoSeg: 130
```

```
},  
  
{  
  producto: 'P15',  
  tiempoSeg: 135  
},  
  
{  
  producto: 'P16',  
  tiempoSeg: 140  
},  
  
{  
  producto: 'P17',  
  tiempoSeg: 145  
},  
  
{  
  producto: 'P18',  
  tiempoSeg: 150  
},  
  
{  
  producto: 'P19',  
  tiempoSeg: 155  
},  
  
{  
  producto: 'P20',  
  tiempoSeg: 160
```

```
},  
  
{  
  producto: 'P21',  
  tiempoSeg: 165  
},  
  
{  
  producto: 'P22',  
  tiempoSeg: 170  
},  
  
{  
  producto: 'P23',  
  tiempoSeg: 175  
},  
  
{  
  producto: 'P24',  
  tiempoSeg: 180  
},  
  
{  
  producto: 'P25',  
  tiempoSeg: 185  
},  
  
{  
  producto: 'P26',  
  tiempoSeg: 190
```

```
},  
  
{  
  producto: 'P27',  
  tiempoSeg: 195  
},  
  
{  
  producto: 'P28',  
  tiempoSeg: 200  
},  
  
{  
  producto: 'P29',  
  tiempoSeg: 205  
}  
]  
}
```

3) Captura de pantalla de comandos ejecutados en MongoDB

- Buscar clientes que tengan un producto favorito específico

```
> db.Clientes.find({productosFavoritos: "P1003"}).pretty()
```

```
< {  
  _id: ObjectId('6a1a67d77dfa6ae657b75404'),  
  nombre: 'Cliente 1',  
  email: 'cliente1@mail.com',  
  direcciones: [  
    {  
      tipo: 'casa',  
      calle: 'Calle 1',  
      ciudad: 'San Salvador'  
    },  
    {  
      tipo: 'trabajo',  
      calle: 'Blvd 1',  
      ciudad: 'Soyapango'  
    },  
    {  
      tipo: 'envío',  
      calle: 'Col 1',  
      ciudad: 'Santa Tecla'  
    }  
  ]  
}
```

```
  ],  
  productosFavoritos: [  
    'P1001',  
    'P1002',  
    'P1003',  
    'P1004',  
    'P1005',  
    'P1006',  
    'P1007',  
    'P1008',  
    'P1009',  
    'P1010'  
  ],  
  metodosPago: [  
    {  
      tipo: 'tarjeta',  
      banco: 'BAC'  
    },  
    {  
      tipo: 'paypal'  
    }  
  ]  
}
```

Buscar

23:09
29/5/2026


```
>_ MONGOSH
banco: 'BAC'
},
{
  tipo: 'paypal'
},
{
  tipo: 'efectivo'
},
{
  tipo: 'transferencia'
},
{
  tipo: 'bitcoin'
}
],
compras: [
  {
    idCompra: 'C10',
    total: 50
  },
  {
    idCompra: 'C11',
    total: 65
  },
  {
```

```
>_ MONGOSH
    idCompra: 'C11',
    total: 65
  },
  {
    idCompra: 'C12',
    total: 80
  },
  {
    idCompra: 'C13',
    total: 95
  },
  {
    idCompra: 'C14',
    total: 110
  },
  {
    idCompra: 'C15',
    total: 125
  },
  {
    idCompra: 'C16',
    total: 140
  },
  {
    idCompra: 'C17',
```

```
idCompra: 'C17',
total: 155
},
{
  idCompra: 'C18',
total: 170
},
{
  idCompra: 'C19',
total: 185
},
{
  idCompra: 'C110',
total: 200
},
{
  idCompra: 'C111',
total: 215
},
{
  idCompra: 'C112',
total: 230
},
},
```

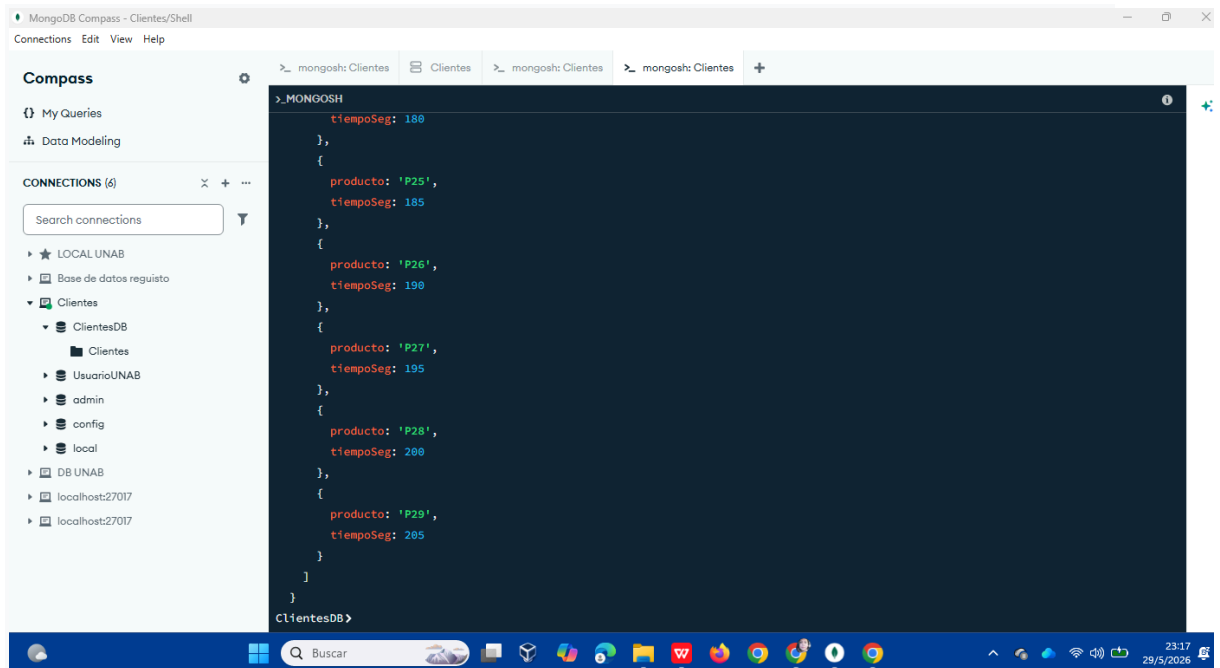
Buscar

23:11
29/5/2026

```
>_MONGOSH
idCompra: 'C112',
total: 230
},
{
  idCompra: 'C113',
total: 245
},
{
  idCompra: 'C114',
total: 260
},
{
  idCompra: 'C115',
total: 275
},
{
  idCompra: 'C116',
total: 290
},
{
  idCompra: 'C117',
total: 305
},
{
  idCompra: 'C118',
total: 320
},
```

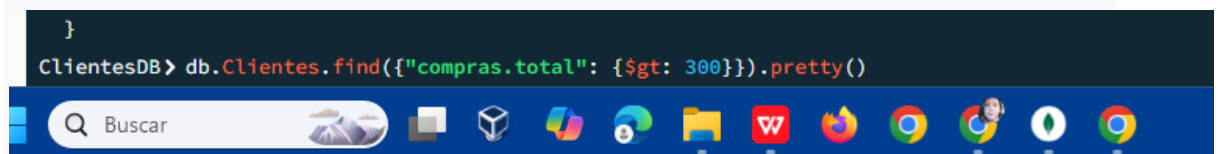
```
>_MONGOSH
{
  },
  {
    idCompra: 'C119',
    total: 335
  }
],
historialNavegacion: [
  {
    producto: 'P0',
    tiempoSeg: 60
  },
  {
    producto: 'P1',
    tiempoSeg: 65
  },
  {
    producto: 'P2',
    tiempoSeg: 70
  },
  {
    producto: 'P3',
    tiempoSeg: 75
  },
  {
    producto: 'P4'
  }
]
```

```
>_MONGOSH
{
  producto: 'P3',
  tiempoSeg: 75
},
{
  producto: 'P4',
  tiempoSeg: 80
},
{
  producto: 'P5',
  tiempoSeg: 85
},
{
  producto: 'P6',
  tiempoSeg: 90
},
{
  producto: 'P7',
  tiempoSeg: 95
},
{
  producto: 'P8',
  tiempoSeg: 100
},
{
  producto: 'P9',
  tiempoSeg: 105
}
```



2. Buscar clientes que tengan compras mayores a \$300:

```
db.Clientes.find({"compras.total": {$gt: 300}}).pretty()
```

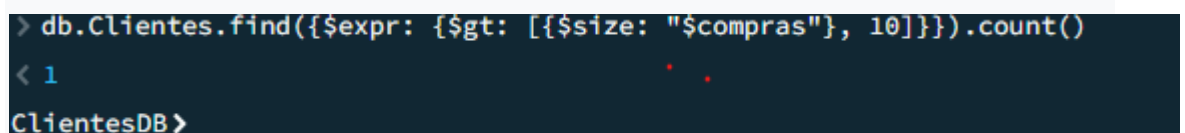


- Buscar clientes que tengan compras mayores a \$300

3. Contar clientes con más de 10 compras:

```
db.Clientes.find({$expr: {$gt: [{$size: "$compras"}, 10]}}).count()
```

Conclusión: El cliente si tiene mas de 10 compras



- Agregar un nuevo producto favorito

```

> db.Clientes.updateOne({email: "cliente1@mail.com"}, {$push: {productosFavoritos: "P9999"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ClientesDB>

```

Conclusión: comprobando producto favorito

```

> db.Clientes.findOne({email: "cliente1@mail.com"}, {productosFavoritos: 1, _id: 0})
< {
  productosFavoritos: [
    'P1001',
    'P1002',
    'P1003',
    'P1004',
    'P1005',
    'P1006',
    'P1007',
    'P1008',
    'P1009',
    'P1010',
    'P9999'
  ]
}
ClientesDB>

```

- **Eliminar un producto favorito**

5. Eliminar un producto favorito - \$pull:

```

db.Clientes.updateOne({email: "cliente1@mail.com"}, {$pull:
{productosFavoritos: "P1005"}})

```

```

db.Clientes.findOne({email: "cliente1@mail.com"}, {productosFavoritos: 1, _id:

```

Captura de pantalla eliminación de productos Favoritos

```

> db.Clientes.updateOne({email: "cliente1@mail.com"}, {$pull: {productosFavoritos: "P1005"}})
db.Clientes.findOne({email: "cliente1@mail.com"}, {productosFavoritos: 1, _id: 0})
< {
  productosFavoritos: [
    'P1001',
    'P1002',
    'P1003',
    'P1004',
    'P1006',
    'P1007',
    'P1008',
    'P1009',
    'P1010',
    'P9999'
  ]
}
ClientesDB>

```

6) Evitar elementos duplicados

Conclusión: En la captura de pantalla observamos que se visualiza que lo deja como estaba y no lo duplicó.

```

> db.Clientes.updateOne({email: "cliente1@mail.com"}, {$addToSet: {productosFavoritos: "P8888"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
ClientesDB>

```

Cada grupo deberá responder:

1. ¿Qué ventajas ofrecen los arreglos en MongoDB?
2. ¿Qué diferencias existen respecto a las tablas relacionales?
3. ¿Cómo ayudan los arreglos a representar información compleja?
4. ¿Qué problemas podrían aparecer con documentos demasiado grandes?
5. ¿En qué escenarios empresariales son más útiles los arreglos?

Desarrollo:

1.1. ¿Qué ventajas ofrecen los arreglos en MongoDB?

Permiten guardar listas de valores en un solo documento sin crear tablas extra. Consultas más rápidas porque traes todo de una.

. ¿Qué diferencias existen respecto a las tablas relacionales?

En SQL necesitás tabla cliente + tabla productos favoritos con JOIN. En MongoDB el array va dentro del mismo documento del cliente.

3. ¿Cómo ayudan los arreglos a representar información compleja?

Podés guardar objetos anidados, historial de compras, tags, etc. Todo estructurado dentro del cliente sin normalizar.

4. ¿Qué problemas podrían aparecer con documentos demasiado grandes?

Límite de 16 MB por documento. Si el array crece infinito, el documento se vuelve lento para leer/actualizar.

5. ¿En qué escenarios empresariales son más útiles los arreglos?

Carrito de compras, lista de favoritos, tags de productos, historial de direcciones. Datos que pertenecen 100% a un solo cliente.

Cada grupo deberá entregar:

- Script `.js` de creación.
- Capturas de las consultas.
- Explicación de resultados.
- **Conclusiones sobre MongoDB y arreglos.**

Desarrollo:

1. Script .js de creación - script.js

// Borrar colección si existe

```
db.Clientes.drop()
```

// Crear 3 clientes con arrays

```
db.Clientes.insertMany([
  {
    email: "cliente1@mail.com",
    nombre: "Ana López",
    productosFavoritos: ["P1001", "P2002"]
  },
  {
    email: "cliente2@mail.com",
    nombre: "Luis Torres",
    productosFavoritos: ["P3003", "P4004", "P5005"]
  },
  {
    email: "cliente3@mail.com",
    nombre: "María Díaz",
    productosFavoritos: []
  }
])
```

- **Capturas de las consultas.**


```
// Paso 1: Agrega P8888 - modifiedCount: 1
db.Clientes.updateOne(
  {email: "cliente1@mail.com"},
  {$addToSet: {productosFavoritos: "P8888"}}
)

// Paso 2: Intenta agregar P8888 de nuevo - modifiedCount: 0 ← Evidencia
db.Clientes.updateOne(
  {email: "cliente1@mail.com"},
  {$addToSet: {productosFavoritos: "P8888"}}
)

// Paso 3: Verifica que no se duplicó
db.Clientes.findOne(
  {email: "cliente1@mail.com"},
  {productosFavoritos: 1, _id: 0}
```

- **Explicación de resultados.**

3. Respuestas a las 5 preguntas

1. ¿Qué ventajas ofrecen los arreglos en MongoDB?

Almacenan múltiples valores en un solo campo sin tablas extra. Permiten consultas atómicas, reducen JOINS y el modelo se adapta mejor a datos como listas, tags o historial.

2. ¿Qué diferencias existen respecto a las tablas relacionales?

En SQL se normaliza: tabla Cliente + tabla Cliente Producto con clave foránea + JOIN. En MongoDB el array va embebido en el mismo documento del cliente. Menos tablas, lectura en 1 sola operación.

3. ¿Cómo ayudan los arreglos a representar información compleja?

Pueden guardar objetos completos, no solo strings. Ej: historial de compras con {producto, fecha, precio}. Permiten modelar relaciones 1-a-muchos directamente dentro del documento padre.

4. ¿Qué problemas podrían aparecer con documentos demasiado grandes?

Límite duro de 16MB por documento en MongoDB. Arrays que crecen sin control = documentos pesados, updates lentos y mayor consumo de RAM.

Solución: usar colecciones separadas si el array será muy grande.

5. ¿En qué escenarios empresariales son más útiles los arreglos?

Carrito de compras, lista de favoritos, tags/categorías de producto, direcciones de envío, historial de login. Todo dato que pertenece 100% a una entidad y se consulta junto con ella.

Explicación de resultados \$addToSet

\$addToSet es el operador que evita duplicados. Funciona como un "SET" matemático.

- 1ra ejecución: P8888 no existe → lo agrega → modifiedCount: 1

- 2da ejecución: P8888 ya existe → no hace nada → modifiedCount: 0

Diferencia con \$push: \$push siempre agrega aunque se repita.

- **Conclusiones sobre MongoDB y arreglos.**

Los arreglos embebidos son una de las fortalezas de MongoDB para modelar datos del mundo real. Simplifican el esquema y mejoran la performance en lecturas cuando los datos relacionados se usan juntos. La clave está en usar \$addToSet para mantener integridad y vigilar el tamaño del documento para no pasar los 16MB.